

Mem Table and Merge Iterators

 This is a legacy version of the Mini-LSM course and we will not maintain it anymore. We now have a better version of this course and this chapter is now part of [Mini-LSM Week 1 Day 1: Memtable](#) and [Mini-LSM Week 1 Day 2: Merge Iterator](#)

- [Task 1 - Mem Table](#)
- [Task 2 - Mem Table Iterator](#)
- [Task 3 - Merge Iterator](#)
- [Task 4 - Two Merge Iterator](#)
- [Extra Tasks](#)

In this part, you will need to modify:

- `src/iterators/merge_iterator.rs`
- `src/iterators/two_merge_iterator.rs`
- `src/mem_table.rs`

You can use `cargo x copy-test day3` to copy our provided test cases to the starter code directory. After you have finished this part, use `cargo x scheck` to check the style and run all test cases. If you want to write your own test cases, write a new module `#[cfg(test)] mod user_tests { /* your test cases */ }` in `table.rs`. Remember to remove `#![allow(...)]` at the top of the modules you modified so that cargo clippy can actually check the styles.

This is the last part for the basic building blocks of an LSM tree. After implementing the merge iterators, we can easily merge data from different part of the data structure (mem table + SST) and get an iterator over all data. And in part 4, we will compose all these things together to make a real storage engine.

Task 1 - Mem Table

In this course, we use [crossbeam-skiplist](#) as the implementation of memtable. Skiplist is like linked-list, where data is stored in a list node and will not be moved in memory. Instead of using a single pointer for the next element, the nodes in skiplists contain multiple pointers and allow user to “skip some elements”, so that we can achieve $O(\log n)$ search, insertion, and deletion.

In storage engine, users will create iterators over the data structure. Generally, once user modifies the data structure, the iterator will become invalid (which is the case for C++ STL and Rust containers). However, skiplists allow us to access and modify the data structure at

the same time, therefore potentially improving the performance when there is concurrent access. There are some papers argue that skiplists are bad, but the good property that data stays in its place in memory can make the implementation easier for us.

In `mem_table.rs`, you will need to implement a mem-table based on crossbeam-skiplist. Note that the memtable only supports `get`, `scan`, and `put` without `delete`. The deletion is represented as a tombstone `key -> empty value`, and the actual data will be deleted during the compaction process (day 5). Note that all `get`, `scan`, `put` functions only need `&self`, which means that we can concurrently call these operations.

Task 2 - Mem Table Iterator

You can now implement an iterator `MemTableIterator` for `MemTable`.

`memtable.iter(start, end)` will create an iterator that returns all elements within the range `start, end`. Here, `start` is `std::ops::Bound`, which contains 3 variants: `Unbounded`, `Included(key)`, `Excluded(key)`. The expressiveness of `std::ops::Bound` eliminates the need to memorizing whether an API has a closed range or open range.

Note that `crossbeam-skiplist`'s iterator has the same lifetime as the skiplist itself, which means that we will always need to provide a lifetime when using the iterator. This is very hard to use. You can use the `ouroboros` crate to create a self-referential struct that erases the lifetime. You will find the [ouroboros examples](#) helpful.

```
pub struct MemTableIterator {
    /// hold the reference to the skiplist so that the iterator will be valid.
    map: Arc<Skiplist>
    /// then the lifetime of the iterator should be the same as the
    `MemTableIterator` struct itself
    iter: Skiplist::Iter<'this>
}
```

You will also need to convert the Rust-style iterator API to our storage iterator. In Rust, we use `next() -> Data`. But in this course, `next` doesn't have a return value, and the data should be fetched by `key()` and `value()`. You will need to think a way to implement this.

► Spoiler: the MemTableIterator struct

In this design, you might have noticed that as long as we have the iterator object, the mem-table cannot be freed from the memory. In this course, we assume user operations are short, so that this will not cause big problems. See extra task for possible improvements.

You can also consider using [AgateDB's skiplist](#) implementation, which avoids the problem of creating a self-referential struct.

Task 3 - Merge Iterator

Now that you have a lot of mem-tables and SSTs, you might want to merge them to get the latest occurrence of a key. In `merge_iterator.rs`, we have `MergeIterator`, which is an iterator that merges all iterators *of the same type*. The iterator at the lower index position of the `new` function has higher priority, that is to say, if we have:

```
iter1: 1->a, 2->b, 3->c
iter2: 1->d
iter: MergeIterator::create(vec![iter1, iter2])
```

The final iterator will produce `1->a, 2->b, 3->c`. The data in `iter1` will overwrite the data in other iterators.

You can use a `BinaryHeap` to implement this merge iterator. Note that you should never put any invalid iterator inside the binary heap. One common pitfall is on error handling. For example,

```
let Some(mut inner_iter) = self.iters.peek_mut() {
    inner_iter.next()?; // <- will cause problem
}
```

If `next` returns an error (i.e., due to disk failure, network failure, checksum error, etc.), it is no longer valid. However, when we go out of the `if` condition and return the error to the caller, `PeekMut`'s `drop` will try move the element within the heap, which causes an access to an invalid iterator. Therefore, you will need to do all error handling by yourself instead of using `?` within the scope of `PeekMut`.

You will also need to define a wrapper for the storage iterator so that `BinaryHeap` can compare across all iterators.

Task 4 - Two Merge Iterator

The LSM has two structures for storing data: the mem-tables in memory, and the SSTs on disk. After we constructed the iterator for all SSTs and all mem-tables respectively, we will need a new iterator to merge iterators of two different types. That is `TwoMergeIterator`.

You can implement `TwoMergeIterator` in `two_merge_iter.rs`. Similar to `MergeIterator`, if the same key is found in both of the iterator, the first iterator takes precedence.

In this course, we explicitly did not use something like `Box<dyn StorageIter>` to avoid dynamic dispatch. This is a common optimization in LSM storage engines.

Extra Tasks

- Implement different mem-table and see how it differs from skiplist. i.e., BTree mem-table. You will notice that it is hard to get an iterator over the B+ tree without holding a lock of the same timespan as the iterator. You might need to think of smart ways of solving this.
- Async iterator. One interesting thing to explore is to see if it is possible to asynchronize everything in the storage engine. You might find some lifetime related problems and need to workaround them.
- Foreground iterator. In this course we assumed that all operations are short, so that we can hold reference to mem-table in the iterator. If an iterator is held by users for a long time, the whole mem-table (which might be 256MB) will stay in the memory even if it has been flushed to disk. To solve this, we can provide a `ForegroundIterator` / `LongIterator` to our user. The iterator will periodically create new underlying storage iterator so as to allow garbage collection of the resources.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.